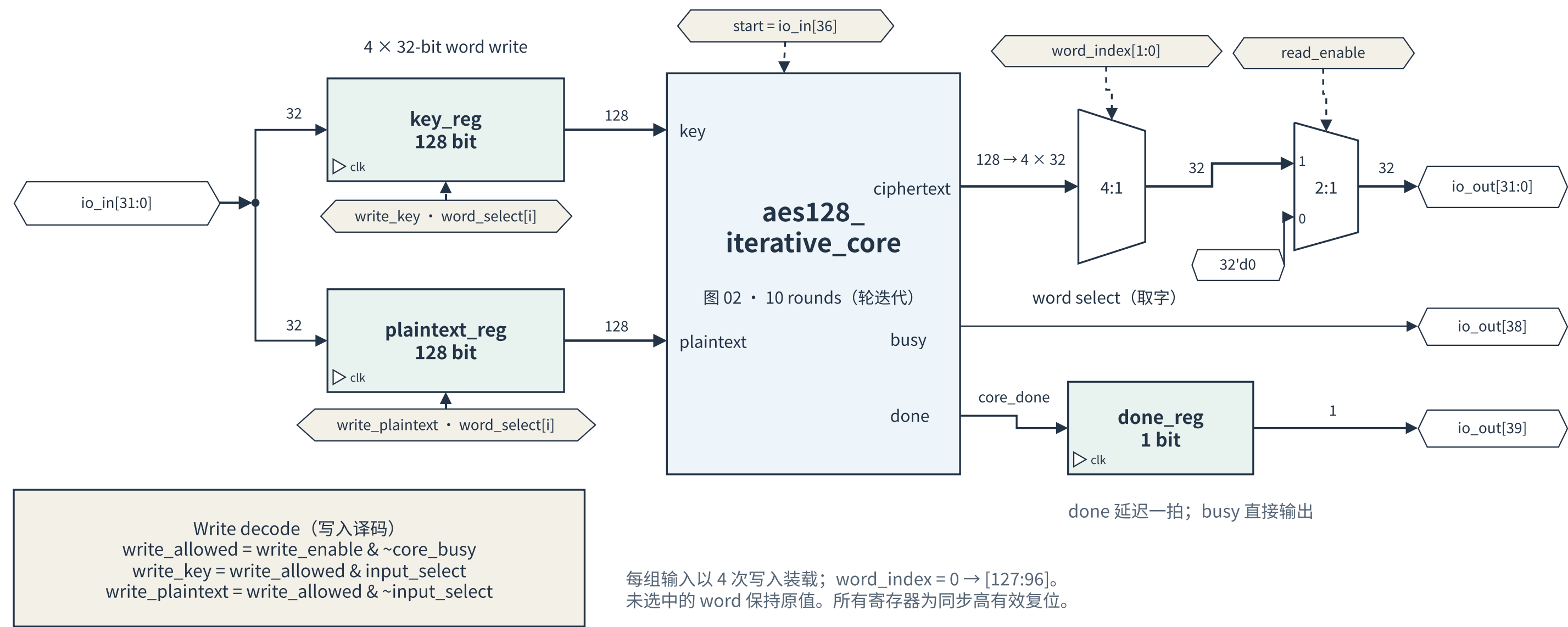


01 Aes128Iterative

32-bit shared IO（共享数据接口）与 AES-128 核心



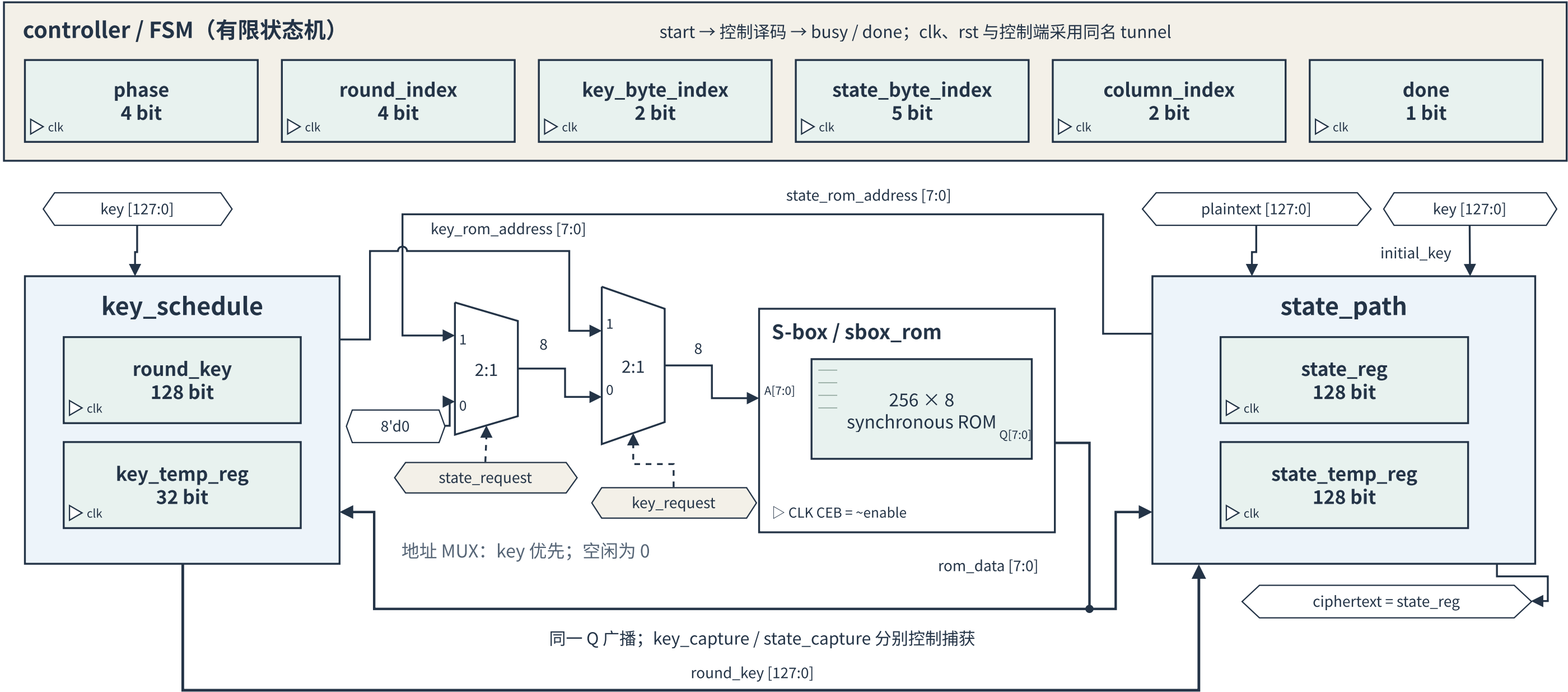
IO mapping（接口映射）

io_in[33:32] = word_index [34] = input_select [35] = write_enable [36] = start [37] = read_enable

io_oe[31:0] = {32{read_enable}} io_oe[39:38] = 2'b11；其余输出及 OE 置 0。IO_WIDTH 默认 66，数据通道为 32 bit。

02 aes128_iterative_core

一个同步 S-box ROM；状态与密钥通路分时共享

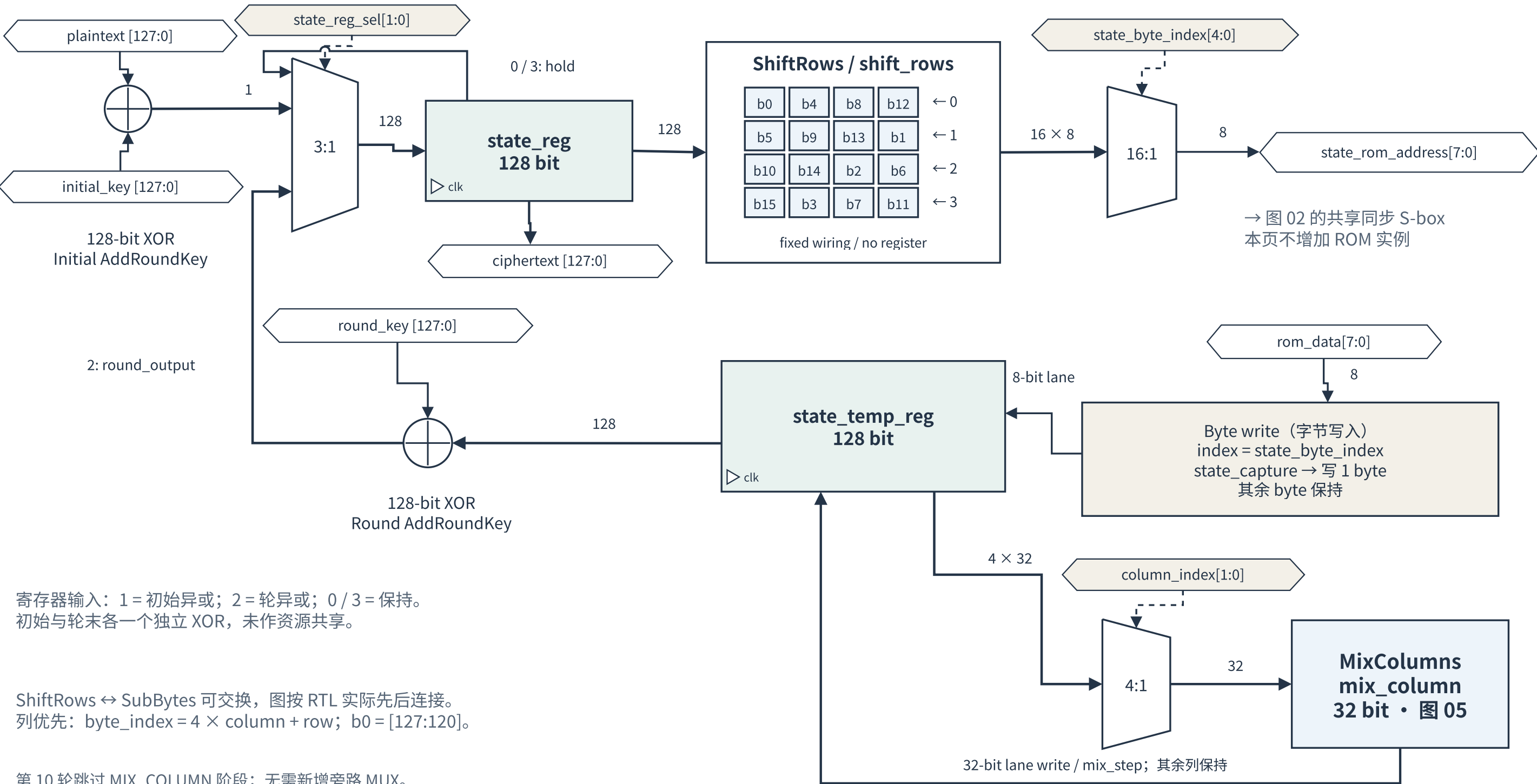


ROM enable = key_request | state_request; CEB = ~enable.
clk 上升沿同步读取; REQUEST 发地址, CAPTURE 写回。

每轮: 4 次密钥查表 → KEY_UPDATE
→ 16 次状态查表 → 4 列 MixColumns (末轮跳过)
→ ADD_KEY; 资源按控制阶段分时复用。

03 state_path

字节串行 SubBytes + 固定 ShiftRows；单列 MixColumns 复用



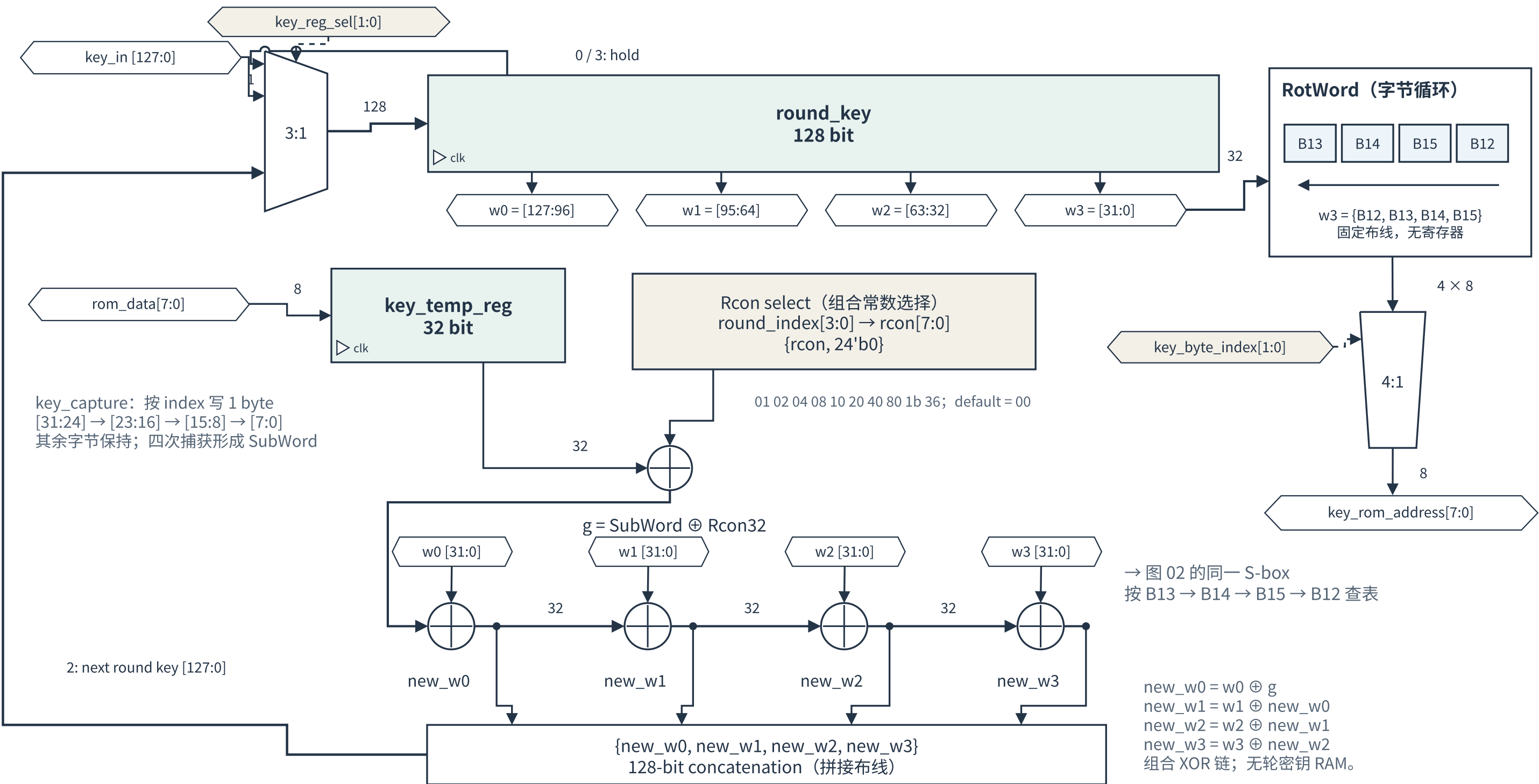
寄存器输入：1 = 初始异或；2 = 轮异或；0 / 3 = 保持。
初始与轮末各一个独立 XOR，未作资源共享。

ShiftRows ↔ SubBytes 可交换，图按 RTL 实际先后连接。
列优先：byte_index = 4 × column + row；b0 = [127:120]。

第 10 轮跳过 MIX_COLUMN 阶段；无需新增旁路 MUX。

04 key_schedule

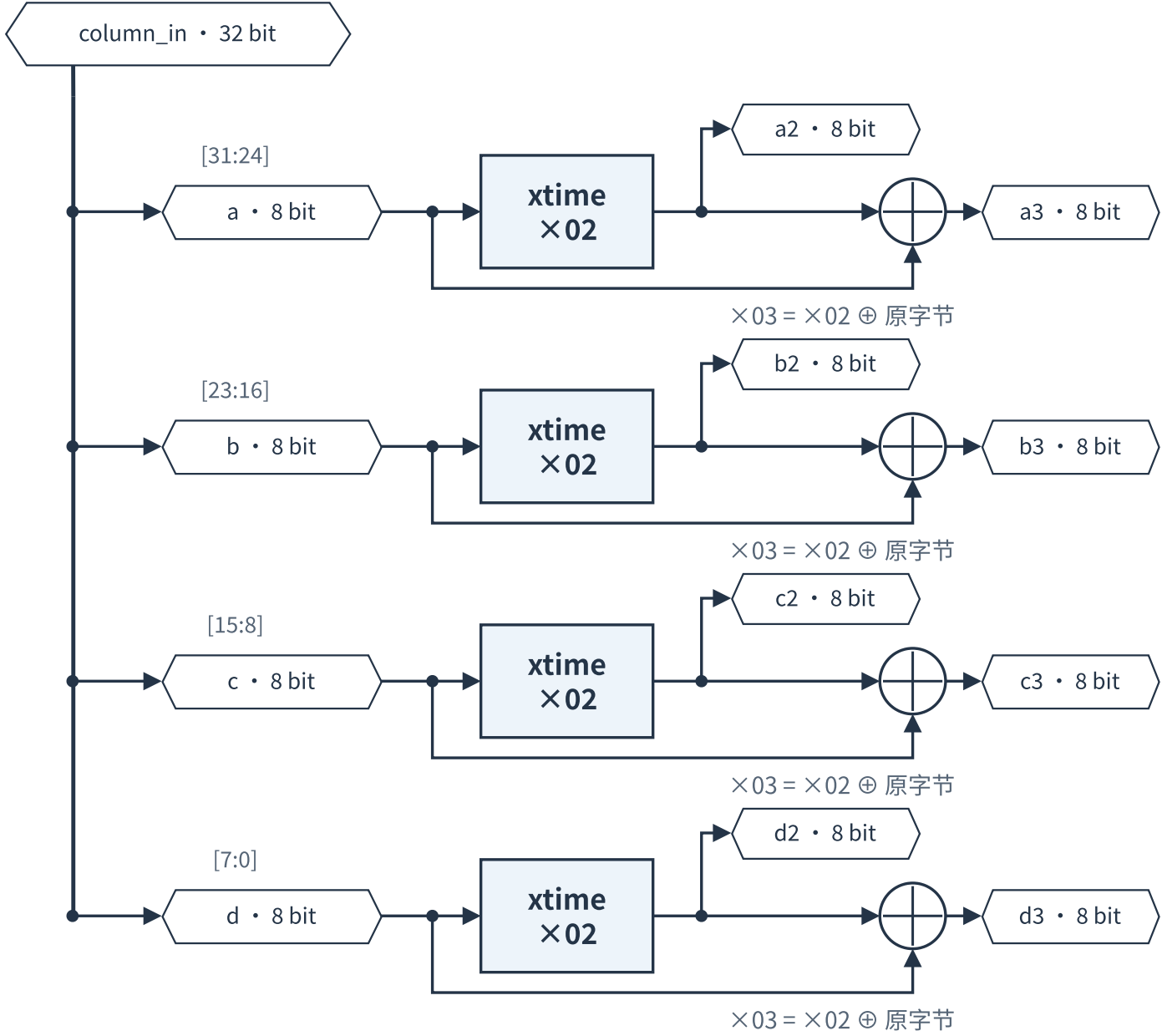
当前轮密钥就地更新；四次共享 S-box 查询完成 SubWord



05 mix_column — 单列组合运算

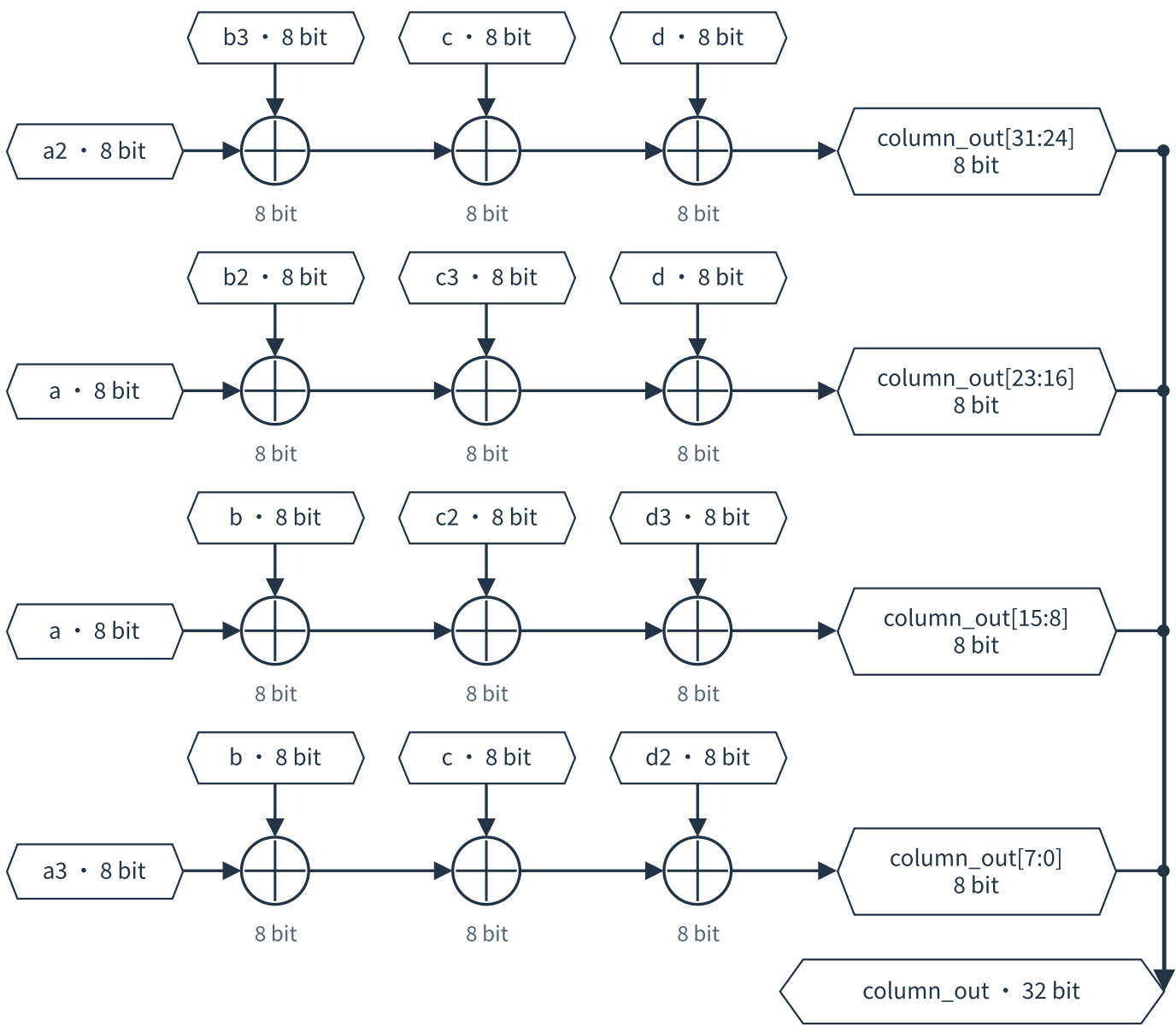
一个 32-bit 列单元在四列间分时复用；单元内部有四路并行 xtime，全部为组合逻辑。

① 字节拆分与常数乘法



GF(2⁸) (8 位有限域)：×02 由 xtime 实现；
×03 由 xtime(x) ⊕ x 实现。xtime 的展开见图 06。

② 按 RTL 表达式形成四个输出字节

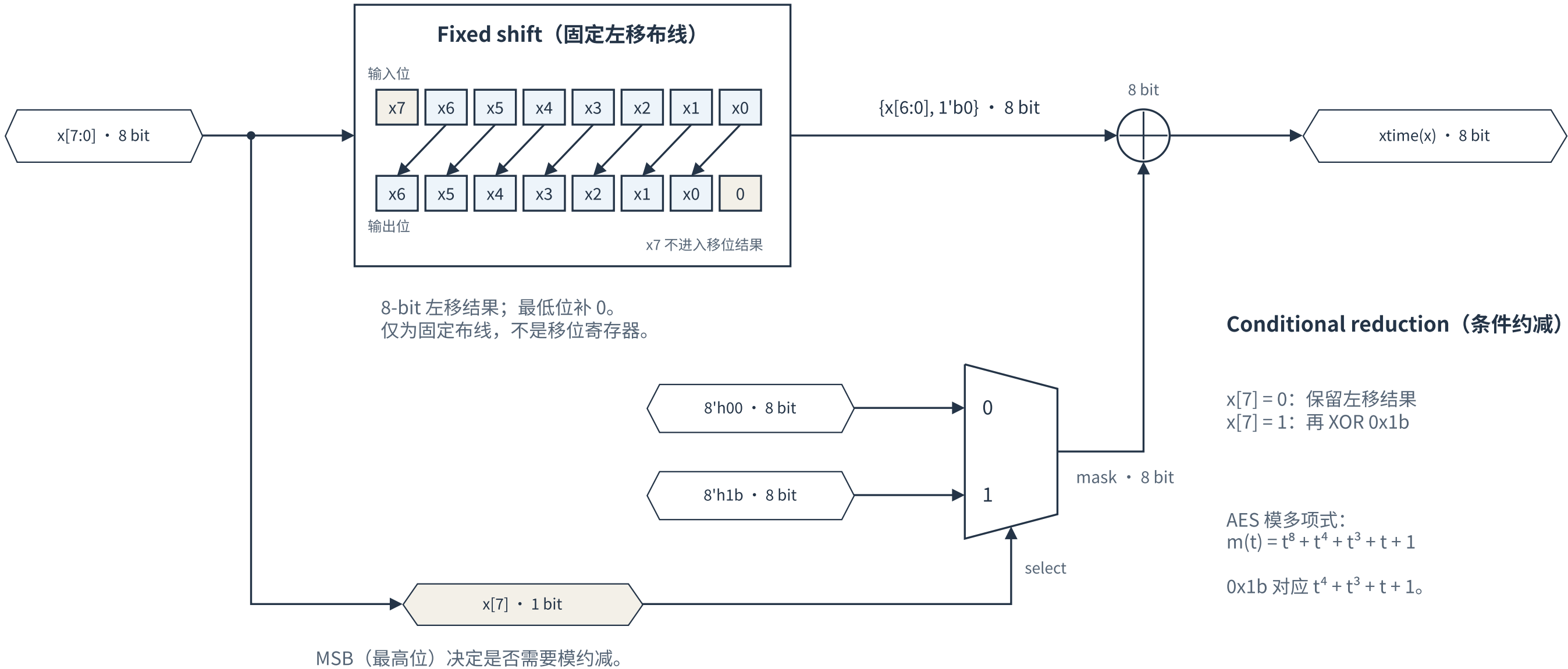


同名 tunnel (网络标签) 连接同一 8-bit 数据网。
三个 XOR 符号表示每个输出的四项异或，未进行公共项优化。

06 xtime — ×02 in GF(2⁸)

固定左移布线与条件模约减；这是 mix_column 内 Verilog function 的图形封装，无寄存器。

$$\text{xtime}(x) = \{x[6:0], 1'b0\} \oplus (x[7] ? 8'h1b : 8'h00)$$



MUX（选择器）是 RTL 中 $8'h1b \& \{8\{x[7]\}\}$ 的等价示意表达；它不是额外的时序级。